A

Project Report

on

Spend Smart: An AI-Powered Personal Finance Management Platform

submitted as partial fulfilment for the award of

BACHELOR OF TECHNOLOGY DEGREE

SESSION 2025-26

in

Computer Science & Engineering (Artificial Intelligence & Machine Learning)

By

Aniket Nimiya (2200291530012)

Aryan (2200291530024)

Aman Chauhan (2200291530008)

Dipendra Yadav (2300291539004)

Under the supervision of

Ms. Surbhi Verma

KIET Group of Institutions, Ghaziabad

Affiliated to

Dr. A.P.J. Abdul Kalam Technical University, Lucknow

(Formerly UPTU)

May, 2026

DECLARATION

We hereby declare that this submission is our own work and that, to the best of our knowledge and belief, it contains no material previously published or written by another person nor material which to a substantial extent has been accepted for the award of any other degree or diploma of the university or other institute of higher learning, except where due acknowledgment has been made in the text.

The project titled "Spend Smart: An AI-Powered Personal Finance Management Platform for Budgeting, Analytics, and Automated Financial Assistance" has been developed by us as students of B.Tech. in Computer Science & Engineering (AI & ML) at KIET Group of Institutions, Ghaziabad. The work presented in this report is genuine and original. All sources of information used in this project have been duly cited and acknowledged.

Aniket Nimiya (2200291530012) Signature: _____

Aryan (2200291530024) Signature: _____

Aman Chauhan (2200291530008) Signature: _____

Dipendra Yadav (2300291539004) Signature: _____

Date: _____

CERTIFICATE

This is to certify that Project Report entitled "Spend Smart: An AI-Powered Personal Finance Management Platform for Budgeting, Analytics, and Automated Financial Assistance" which is submitted by Aniket Nimiya (2200291530012), Aryan (2200291530024), Aman Chauhan (2200291530008), and Dipendra Yadav (2300291539004) in partial fulfillment of the requirement for the award of degree B. Tech. in Department of CSE (AI & ML) of Dr. A.P.J. Abdul Kalam Technical University, Lucknow is a record of the candidates' own work carried out by them under my supervision. The matter embodied in this report is original and has not been submitted for the award of any other degree.

Ms. Surbhi Verma **Dr. Rekha Kashyap**

(Assistant Professor, CSE(AI&ML)) **Dean – CSE (AI/AI&ML)**

Date:

ACKNOWLEDGEMENT

It gives us a great sense of pleasure to present the report of the B. Tech Project undertaken during B. Tech. Final Year. We owe special debt of gratitude to Ms. Surbhi Verma, Department of CSE(AI&ML), KIET, Ghaziabad, for his constant support and guidance throughout the course of our work. Her sincerity, thoroughness and perseverance have been a constant source of inspiration for us. It is only her cognizant efforts that our endeavors have seen light of the day.

We also take the opportunity to acknowledge the contribution of Dr. Rekha Kashyap, Dean of the Department of Computer Science & Engineering (AI/AI&ML), KIET, Ghaziabad, for her full support and assistance during the development of the project. We also do not like to miss the opportunity to acknowledge the contribution of all the faculty members of the department for their kind assistance and cooperation during the development of our project.

Date:

Aniket Nimiya (2200291530012) Signature: _____

Aryan (2200291530024) Signature: _____

Aman Chauhan (2200291530008) Signature: _____

Dipendra Yadav (2300291539004) Signature: _____

ABSTRACT

Modern personal finance management applications must move beyond traditional manual expense tracking by offering automation, intelligent insights, and better visibility into financial activities. This project presents Spend Smart, a full-stack AI-powered web application designed to help users efficiently manage their personal finances. The system allows users to record income and expenses, categorize transactions, create monthly budgets, track savings goals, and monitor financial habits through an interactive dashboard.

Spend Smart is developed using a modern technology stack consisting of Next.js and React for the frontend, Prisma ORM for database management, and Supabase for PostgreSQL hosting and cloud file storage. Secure authentication is implemented through Clerk, while Arcjet middleware provides rate limiting and bot protection. React Hook Form and Zod are used for data validation to ensure accuracy and consistency.

Artificial intelligence is integrated using Google Gemini, which provides automated receipt scanning and personalized financial summaries. Users can upload receipt images, and the AI extracts details such as merchant name, date, purchased items, and total amount, reducing manual data entry. Monthly financial data is processed through Inngest background workflows, which generate personalized summaries and deliver them via email using Resend and React Email templates.

For visualization, Recharts is used to generate interactive bar charts, pie charts, and line graphs showing income, expenses, spending categories, and savings trends. Inngest also handles recurring transactions, budget alerts, and scheduled reporting tasks efficiently.

Spend Smart improves upon conventional finance trackers by combining automation, AI intelligence, security, and data visualization into one platform. Future enhancements include Open Banking API integration, UPI transaction monitoring, predictive analytics, and conversational AI support .

.

TABLE OF CONTENTS **Page No.**

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
CSS	Cascading Style Sheets
DBMS	Database Management System
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
ML	Machine Learning
OAuth	Open Authorization
ORM	Object-Relational Mapping
SQL	Structured Query Language
URL	Uniform Resource Locator
UI	User Interface
UX	User Experience

CHAPTER 1

INTRODUCTION

1.1 Introduction

Personal finance management tools based on digital technologies have progressed significantly from applications primarily devoted to the recording of financial transactions toward more complex software systems that allow for budgeting, forecasting, behavioral analysis, and proactive financial advice. In today's fast-paced economy, individuals face increasing complexity in managing their income, expenditures, savings goals, and recurring financial obligations across multiple accounts and categories.

The proliferation of smartphones, cloud computing, and AI-based services has opened up new possibilities for building intelligent financial assistant platforms that can not only store and organize financial data but also interpret it, identify patterns, generate predictions, and provide personalized guidance. Despite these technological advances, the majority of widely used personal finance tools still fall short in terms of automation, intelligence, and ease of use.

Traditional expense trackers and budgeting apps require repetitive manual data entry, offer limited analytical capabilities, and do not leverage the power of modern AI to provide contextual, personalized financial insights. Users are left with raw data, basic charts, and static reports that do not translate into actionable recommendations or intelligent assistance. This gap between what is technically possible and what is delivered in practice is the core motivation behind the development of Spend Smart.

Spend Smart is an AI-powered, full-stack personal finance management web application that addresses all these shortcomings. It integrates intelligent automation, data visualization, AI-driven insights, and robust security into a single, cohesive, and user-friendly platform. The system goes beyond simple record-keeping to become a true digital financial assistant one that learns from user behavior, automates repetitive tasks, and communicates meaningful insights through natural language and visual representations.

1.2 Project Description

Spend Smart is a web application built for the modern individual who seeks a smarter, more automated approach to personal finance management. The application enables users to create and manage multiple financial accounts (such as savings, checking, and credit card accounts), record income and expense transactions, set up monthly budgets per spending category, and track their savings progress all from a single unified interface.

The project is developed as a full-stack web application using Next.js and React for the frontend, with Prisma ORM and Supabase PostgreSQL for backend data management. Financial data security is ensured through Clerk's OAuth-based authentication and Arcjet's middleware-level abuse protection. Data validation is strictly enforced using Zod schemas integrated with React Hook Form to prevent corrupted or malformed financial data from entering the system.

One of the most distinctive features of Spend Smart is its AI-powered receipt scanning capability. When a user uploads an image of a physical or digital receipt, the application encodes it in base64 and submits it to the Google Gemini AI model (gemini-1.5-pro) along with a structured extraction prompt. Gemini returns a JSON object containing the merchant name, transaction date, individual line items, and total amount, which is used to auto-populate the transaction entry form — dramatically reducing the time and effort required to log purchases.

At the close of each calendar month, Spend Smart automatically generates a personalized financial report for every user. An Inngest-triggered background job aggregates the month's transaction data, computes spending by category, calculates savings rate, and compares results with the previous month. This structured data is sent to Gemini, which generates a comprehensive, human-readable financial narrative that is embedded in a polished React Email template and delivered via the Resend email service.

The application's dashboard is the central interface through which users interact with their financial data. Powered by the Recharts library, it displays real-time charts including income vs. expenses bar charts, category-wise spending pie charts, savings balance line graphs, and budget

consumption progress bars. These visualizations are rendered through Next.js server components, ensuring fast, server-rendered chart data without client-side fetch delays.

1.3 Problem Statement

Despite the availability of numerous financial management applications, a majority of users continue to struggle with effective personal finance management. The core problems that Spend Smart aims to solve are:

- Manual Data Entry Burden: Most existing applications require users to manually log every transaction, including date, amount, category, merchant, and notes. This is a time-consuming and error-prone process that leads to low adoption and inconsistent data quality over time.
- Lack of Intelligent Analysis: Traditional expense trackers provide transactional records but fail to translate raw data into meaningful, personalized insights. Users are left to interpret their own financial trends without guidance.
- Absence of Automation: Monthly tasks such as generating financial reports, sending budget alerts, and processing recurring bills are not automated in most tools, requiring significant manual effort from users.
- Fragmented Tools: Users often rely on a combination of spreadsheets, banking apps, and manual records to manage their finances, leading to fragmentation and an incomplete picture of their financial health.
- Security Vulnerabilities: Many lightweight financial apps lack robust security measures, leaving sensitive financial data vulnerable to unauthorized access, bot attacks, and data breaches.
- Poor User Experience: Complex interfaces, slow loading times, and non-responsive designs prevent users from engaging consistently with their financial data.

1.3 Objectives Of The Project

The following objectives guide the design and development of Spend Smart:

- To develop a user-friendly platform that makes recording and categorizing income and expenses effortless, reducing the manual effort required by users.

- To implement comprehensive budgeting tools that allow users to define monthly spending limits per category and monitor real-time budget consumption.
- To integrate Google Gemini AI for intelligent receipt scanning, enabling automatic extraction and population of transaction data from uploaded receipt images.
- To provide data-driven financial insights through interactive dashboards that display income, expenses, savings, and budget consumption using visually compelling charts.
- To automate recurring financial workflows including monthly report generation, budget threshold alerts, and recurring transaction processing using Inngest background jobs.
- To ensure the highest level of data security and user privacy through layered security mechanisms including OAuth authentication, rate limiting, bot detection, and database-level row security policies.
- To deliver personalized monthly financial summaries via email, generated by AI and formatted with React Email templates, helping users stay informed about their financial progress.

1.4 Scope Of The Project

The scope of Spend Smart encompasses all aspects of personal financial management for individual users. The platform supports financial account management, transaction tracking, budget planning, savings monitoring, and AI-assisted receipt processing. It is designed as a web-based application accessible through modern desktop and mobile browsers.

The current version of Spend Smart focuses on individual users and does not include multi-user household budgeting or business financial management features. Bank account integration through Open Banking APIs is outside the current scope but is identified as a key area for future development. The AI features are powered by Google's Gemini model through the official Generative AI SDK, with plans to extend these capabilities in future iterations.

The project is relevant to students, working professionals, and anyone seeking to improve their financial awareness and management habits. Its applicability extends across demographic groups, making it a broadly useful tool in the domain of personal financial wellness technology.

CHAPTER 2
LITERATURE REVIEW

2.1 Overview Of Personal Finance Management Systems

Rahmah Mahdi et al. (2024)

This study presents a basic personal expense tracking system focused on recording daily financial activities. It emphasizes simplicity and user-friendly design for individuals. However, it lacks advanced analytics or predictive features.

Sneha Gupta et al. (2024)

The authors developed an expense tracker that categorizes spending and provides summarized reports. The system improves financial awareness through structured data visualization. It mainly focuses on manual input without automation.

Dr. Nidhi Sharma & Dr. Alok Sharma et al. (2024)

This paper introduces smart expense tracking using modern technologies for better financial planning. It highlights automation and efficient data handling. The system still has limited AI-based decision-making capabilities.

Tushar Khuteta et al. (2025)

The Expense Tracker System (ETS) focuses on managing income and expenses efficiently. It provides basic features like data storage and retrieval. The system lacks real-time insights and intelligent recommendations.

A. Saraboji & Dr. Santhana Lakshmi et al. (2024)

This application offers a structured way to track financial transactions and monitor spending habits. It ensures accuracy and easy access to financial records. However, it does not incorporate machine learning techniques.

Lavesh Lingayat et al. (2024)

This work integrates machine learning into expense tracking for real-time analysis. It enhances prediction and categorization of expenses. The system shows improvement over traditional trackers but requires large datasets.

Pooja S. Saravade et al. (2022)
The study presents a daily income-expense tracking system for routine financial management. It focuses on simplicity and consistency in data entry. The system lacks advanced visualization and automation features.

Mohammed Sahel et al. (2024)
This research explores a web-based expense tracker for managing personal finances efficiently. It improves accessibility and user interaction through online platforms. However, it has limited intelligent insights.

Multiple Authors et al. (2025)
This paper proposes an expense tracker using OCR and Random Forest algorithm. It automates data entry by extracting information from receipts. The approach improves accuracy but increases system complexity.

Satyam Jha et al. (2025)

The authors developed a web-based expense tracker for effective financial management. It includes features like categorization and reporting. The system is scalable but lacks advanced predictive analytics.

2.2 AI-Based Financial Applications

The integration of artificial intelligence into personal finance management represents the most significant recent trend in this domain. AI-powered tools are no longer limited to categorization they are now capable of generating natural language summaries, scanning and interpreting physical receipts, predicting future spending patterns, and providing personalized financial coaching.

Lavesh Lingayat et al. (2024) in their paper "Design and Implementation of Real-Time Expense Tracker Using Machine Learning," presented at the ICICC Conference, demonstrated the effectiveness of machine learning models in real-time transaction categorization. Their system achieved a 91% accuracy rate in automatic expense categorization using a Random Forest classifier trained on annotated transaction datasets. However, their system did not incorporate natural language generation capabilities or receipt scanning, areas where Spend Smart provides significant advancement.

The study "Expense Tracker using OCR and Random Forest Algorithm" published in IJERST (2025) explored the combination of Optical Character Recognition and machine learning for automated receipt processing. The researchers reported that OCR-based receipt scanning achieved approximately 78% accuracy for printed receipts. Spend Smart improves upon this approach by using Google's Gemini large language model, which combines visual understanding with language reasoning to achieve significantly higher extraction accuracy, particularly for structured receipt formats.

Mohammed Sahel et al. (2024) in "Streamlining Personal Finances: An Expense Tracker Website Study" presented at IEEE SPARC Conference analyzed multiple expense tracking websites and concluded that the most effective AI integrations in financial applications are those that directly reduce the user's data entry workload. Their findings strongly validate Spend Smart's approach of using AI primarily for receipt scanning and report generation — the two most labor-intensive aspects of personal finance management.

2.3 Dashboard Design and Financial Visualization

Effective data visualization is a critical component of any personal finance system. Users need to be able to glance at their financial position and instantly understand their income, spending, and saving trends without having to interpret raw numbers. Research in dashboard design specifically for financial applications provides important guidelines for how visualization components should be structured, prioritized, and presented.

Research on financial dashboard design emphasizes that effective interfaces should highlight trends, exceptions, and comparisons rather than simply presenting transaction lists. Studies recommend a hierarchical information architecture where summary metrics (total income, total expenses, net savings) are displayed at the top level, followed by categorical breakdowns and temporal trend analyses at deeper navigation levels. Spend Smart's dashboard architecture directly follows these recommendations, providing at-a-glance financial summaries alongside drill-down chart visualizations.

Satyam Jha et al. (2025) in "Design and Implementation of a Web-Based Expense Tracker System for Personal Finance Management" published on Zenodo emphasized the importance of responsive design in financial applications, noting that a majority of users access their financial data on mobile devices. Their study found that applications with poor mobile responsiveness had significantly lower user retention rates. Spend Smart addresses this concern through Tailwind CSS's responsive utility classes and mobile-optimized chart rendering through Recharts.

2.4 Security In Financial WEB Applications

Security is of paramount importance in any application that handles sensitive financial data. The literature on web application security for financial systems identifies authentication, authorization, input validation, and abuse prevention as the four primary security pillars that must be addressed.

A. Saraboji and Dr. Santhana Lakshmi (2024) in their paper "Expense Tracker Application" published in IARJSET examined the security vulnerabilities commonly found in expense tracking applications, including SQL injection, cross-site scripting, insecure direct object references, and broken authentication mechanisms. Their analysis found that many personal finance applications deployed minimal security measures, relying solely on password-based authentication without rate limiting or input sanitization.

The proliferation of bot attacks and automated credential stuffing attacks against financial applications has been documented in multiple recent studies. Research recommends a multi-layered approach combining rate limiting, CAPTCHA, and behavioral analysis to detect and block automated threats. Spend Smart's use of Arcjet middleware for bot detection and rate limiting

directly addresses these recommendations, providing a defense-in-depth security posture that is unusual for individual project-scale financial applications.

2.5 Research Gaps and Motivation

A comprehensive review of the existing literature reveals several critical gaps that motivated the development of Spend Smart:

- Most existing academic and commercial expense trackers provide either AI features or comprehensive automation, but rarely both in a single integrated platform.
- Receipt scanning implementations documented in the literature are primarily based on traditional OCR, which struggles with varied receipt formats. The use of LLM-based vision models like Gemini for receipt extraction represents a novel advancement in this domain.
- Automated natural language financial report generation delivered via email on a monthly schedule is absent from the reviewed related work. Spend Smart introduces this as a core feature.
- The integration of event-driven background processing (via Inngest) for financial automation in web applications is an emerging architectural pattern not yet documented in personal finance research literature.
- Multi-layer security in personal finance applications — combining OAuth authentication, middleware rate limiting, Zod schema validation, and database-level row security — is rarely implemented comprehensively in academic projects.

These gaps establish Spend Smart as a novel contribution to the domain of AI-powered personal finance management, combining state-of-the-art technologies in a manner that addresses the most significant shortcomings identified in existing literature.

CHAPTER 3
PROPOSED METHODOLOGY

3.1 System Design Approach

The development strategy of Spend Smart follows a feature-driven, iterative design approach grounded in user-centered design principles. The methodology began with a thorough requirements analysis phase, during which common tasks performed in financial applications were identified and prioritized. These tasks include recording income and expenses, categorizing transactions, monitoring budgets, analyzing trends, generating financial insights, and automating recurring workflows.

A modular architecture was chosen to ensure that each functional area of the application could be developed, tested, and deployed independently. This modularity also facilitates future enhancements and third-party integrations without disrupting existing functionality. The system is designed around four architectural layers: the Presentation Layer, the Application Logic Layer, the Data and Storage Layer, and the Background Automation Layer.

The design process followed an Agile-inspired iterative cycle, with each sprint focused on delivering a complete, working feature module. This approach allowed for continuous user feedback incorporation and ensured that the most critical features transaction management, budget tracking, and AI integration were developed and validated early in the project lifecycle.

3.2 Technology Stack Selection

The selection of technologies for Spend Smart was guided by three primary criteria: developer productivity, production readiness, and ecosystem maturity. The following table summarizes the technology stack and the rationale for each component:

TABLE 3.1. Technology Stack Summary

Component	Technology	Rationale
Styling	Tailwind CSS + shadcn/ui	Utility-first CSS, pre-built accessible components, design consistency
UI Primitives	Radix UI	Accessible, unstyled components for complex UI patterns
Form Handling	React Hook Form	Performant uncontrolled form management, minimal re-renders
Data Validation	Zod	TypeScript-first schema validation, type inference
ORM	Prisma	Type-safe database access, migration management, schema-first design
Database	Supabase (PostgreSQL)	Managed PostgreSQL, Row-Level Security, cloud file storage
Authentication	Clerk	Complete auth solution with OAuth, MFA, session management
Abuse Protection	Arcjet	Rate limiting, bot detection, middleware integration
AI Services	Google Gemini (generative-ai SDK)	Receipt scanning, financial report generation
Charts	Recharts	React-native charting, interactive tooltips, responsive
Background Jobs	Inngest	Event-driven serverless functions, cron scheduling
Email Service	Resend + React Email	Reliable transactional email, React-based template authoring
Icons	Lucide React	Consistent icon library with React component API

3.3 System Architecture

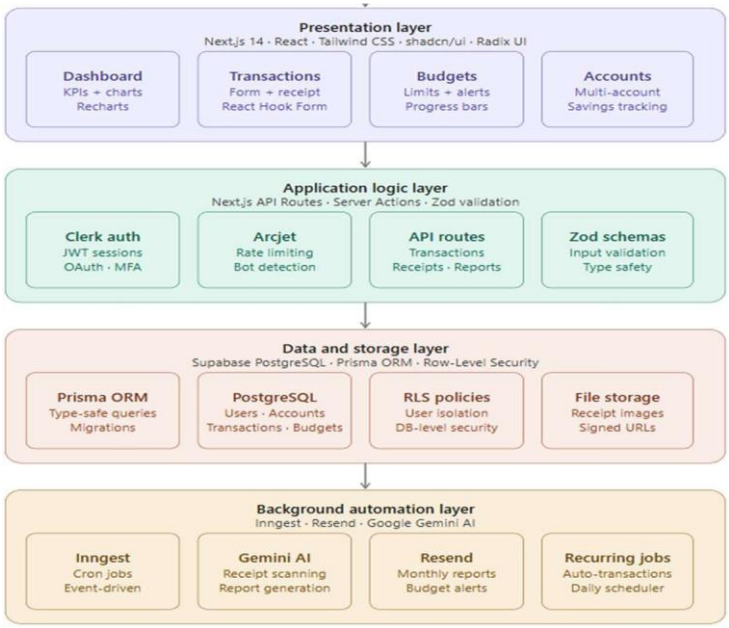


Figure 3.3. Full-Stack System Architecture of Spend Smart

Spend Smart employs a full-stack layered architecture consisting of four primary layers, each with clearly defined responsibilities. This separation of concerns ensures clean code organization, independent scalability, and easier maintenance. The four layers are: (1) Presentation Layer, (2) Application Logic Layer, (3) Data and Storage Layer, and (4) Background Automation Layer.

TABLE 3.3. System Layers and Responsibilities

Layer	Technology	Responsibility
Presentation	Next.js, React, Tailwind, shadcn	UI rendering, user interaction, data display
Application Logic	Next.js API Routes, Prisma, Clerk	Business logic, request handling, service integration
Data & Storage	Supabase PostgreSQL, Supabase Storage	Persistent data storage, file management
Background Automation	Inngest, Resend, Gemini	Async jobs, scheduled tasks, email delivery

3.4 Presentation Layer

The Presentation Layer is built entirely with React and Next.js, leveraging the Next.js App Router for file-based routing and Server Components that render on the server for improved initial page load performance. Client Components handle interactivity, state management, and event-driven user interactions.

Tailwind CSS implements a utility-first approach to styling across the entire application, ensuring visual consistency and responsive design with minimal custom CSS. The shadcn/ui component library provides a comprehensive set of pre-built, accessible UI components built on top of Radix UI primitives. These components include form elements, dialog boxes, dropdown menus, tabs, progress bars, and card layouts all customized to match Spend Smart's design language.

Lucide React provides a consistent icon system that is used throughout the application for navigation elements, financial form fields, dashboard widgets, and status indicators. The Recharts library handles all data visualization in the Presentation Layer, rendering interactive bar charts, pie charts, line graphs, and area charts using SVG-based rendering that scales correctly on all screen sizes.

3.5 Application Logic Layer

The Application Logic Layer is implemented through Next.js server-side methods and API route handlers. These handlers serve as the interface between the Presentation Layer and the Data Layer, processing incoming requests, applying business logic, invoking external services, and returning structured responses.

All incoming data from form submissions and API requests is first validated against Zod schemas before any database operations are performed. This strict validation boundary prevents malformed or malicious data from propagating into the database, ensuring data integrity at all times. The Clerk middleware is applied to all protected routes, ensuring that unauthenticated requests are rejected before reaching business logic code.

External service integrations including Clerk for authentication, Arcjet for abuse protection, and the Gemini AI API for receipt scanning are all invoked from the Application Logic Layer. This centralized integration point ensures that all external communications are properly authenticated, rate-limited, and error-handled.

3.6 Data and Storage Layer

Spend Smart's data persistence layer is built on PostgreSQL, hosted and managed through Supabase. The Prisma ORM provides a type-safe interface for all database operations, with automatically generated TypeScript types derived from the Prisma schema definition. This type safety extends from the database schema all the way through to the API handlers and frontend components, significantly reducing runtime type errors.

The primary data entities managed by the Data Layer include User profiles, Financial Accounts, Transactions, Budget definitions, and Recurring Transaction schedules. Relationships between these entities are defined in the Prisma schema with appropriate foreign key constraints and cascade delete rules to ensure referential integrity.

Supabase's cloud storage is used for receipt image management. When a user uploads a receipt for AI scanning, the image is stored in a Supabase storage bucket and a signed URL is generated. This URL is then passed to the Gemini API for analysis, after which the transaction data is stored in the database with a reference to the receipt image URL.

Supabase's Row-Level Security (RLS) policies are configured to enforce strict data isolation between users, ensuring that a user's financial data is never accessible to another user at the database level, regardless of application-level logic.

3.7 Security Architecture

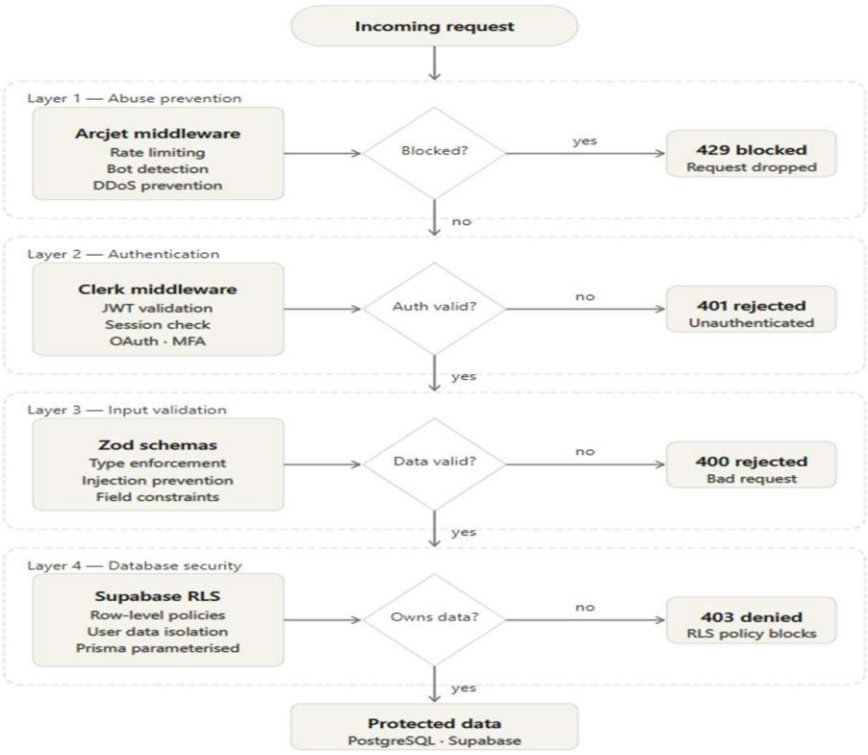


Figure 3.7. Security Architecture Diagram

Spend Smart implements a defense-in-depth security strategy with multiple independent layers of protection:

- Authentication Layer: Clerk provides JWT-based session management, OAuth2 integration with Google, email/password authentication with secure credential storage, and multi-factor authentication support. All protected routes are guarded by Clerk's Next.js middleware, which validates session tokens on every request.
- Abuse Prevention Layer: Arcjet middleware is deployed at the Next.js edge to enforce per-route rate limiting, block known bot user agents, detect credential stuffing patterns, and prevent DDoS-style request floods. Stricter rate limits are applied to sensitive routes such as receipt uploading and AI report generation.
- Input Validation Layer: All API endpoints validate incoming request bodies against Zod schemas before executing any business logic. This prevents injection attacks, type confusion vulnerabilities, and data corruption from malformed inputs.

- Database Security Layer: Supabase Row-Level Security policies ensure that each user can only read and write their own financial data. Even if application-level authorization logic were somehow bypassed, the database would still enforce user-level data isolation.
- File Storage Security: Receipt images in Supabase storage are access-controlled through signed URLs that expire after a configurable time window, preventing unauthorized access to uploaded financial documents.

3.8 Frontend Implementation

The frontend of Spend Smart is implemented as a Next.js 14 application using the App Router, which provides a file-system-based routing model where each folder in the app/ directory corresponds to a route segment. This structure allows for clean organization of page components, layouts, error boundaries, and loading states across the application.

Next.js Server Components are used for data-heavy pages like the dashboard and transaction history, where financial data is fetched directly from the database on the server and sent to the client as pre-rendered HTML. This approach eliminates the need for client-side data fetching waterfalls and significantly improves the Time to First Contentful Paint (TTFCP) for data-rich pages.

Client Components are used for interactive elements such as transaction forms, budget editing interfaces, chart filtering controls, and any UI element that requires local state management or browser-side event handling. The distinction between Server and Client Components is carefully managed throughout the application to achieve the optimal balance between server-side performance and client-side interactivity.

The shadcn ui component library provides the foundational UI building blocks. Components such as Card, Button, Input, Select, Dialog, Tabs, Progress, and Badge are extensively used throughout the application. Each shadcn component is built on Radix UI primitives, ensuring full keyboard accessibility and ARIA compliance. The components are styled using Tailwind CSS class variants, allowing for consistent theming and easy customization.

3.8.1 Dashboard Implementation

The dashboard is the most complex page in the application, aggregating data from multiple database queries and rendering it through four primary chart types. The dashboard layout is organized using a CSS Grid layout with responsive column configurations that adapt between single-column (mobile), two-column (tablet), and three-column (desktop) layouts using Tailwind's responsive breakpoint classes.

The four chart components on the dashboard IncomeExpenseChart, CategoryPieChart, SavingsLineChart, and BudgetProgressSection are each implemented as independent React components that receive pre-fetched data as props. This architecture allows each chart to be rendered independently and updated without re-rendering the entire dashboard. Chart data is fetched server-side and passed to Client Components that handle the Recharts rendering.

3.8.2 Transaction Form Implementation

The transaction entry form is implemented using React Hook Form with a Zod resolver that connects the form state management directly to Zod's schema validation. The form includes fields for transaction type (income/expense), account selection, category selection, amount, date, merchant name, notes, and an optional receipt image upload.

The receipt upload field triggers the AI scanning workflow when an image is selected. The file is immediately uploaded to Supabase Storage, and the resulting signed URL is sent to the Gemini API endpoint via a Next.js API route. The API response containing extracted transaction fields is used to update the form state via React Hook Form's setValue method, auto-populating the remaining form fields without requiring user intervention.

3.9 Backend and API Implementation

The backend of Spend Smart is implemented as a collection of Next.js API Routes located in the app/api/ directory. These routes handle all client-server communication, acting as the interface

between the frontend components and the database, external AI services, and authentication middleware.

Key API routes implemented in Spend Smart include:

TABLE 3.9 API Routes and Their Functions

API Route	Method	Function
/api/transactions	GET	Fetch paginated transaction list with filters
/api/transactions	POST	Create new transaction with Zod validation
/api/transactions/[id]	PUT/DELETE	Update or delete specific transaction
/api/accounts	GET/POST	Manage financial accounts
/api/receipts/scan	POST	Upload receipt and trigger Gemini extraction
/api/reports/monthly	GET	Fetch aggregated monthly report data
/api/recurring	GET/POST	Manage recurring transaction schedules

All API routes validate the session token from the request headers using Clerk's auth() helper function. Unauthenticated requests receive a 401 response before any database queries are executed. The Arcjet rate limiting middleware is applied to sensitive endpoints through Next.js middleware configuration.

3.10 AI Features Implementation

Spend Smart integrates Google's Gemini generative AI through the @google/generative-ai SDK. The AI features are implemented in two primary workflows: receipt scanning and monthly financial report generation.

3.10.1 Receipt Scanning Implementation

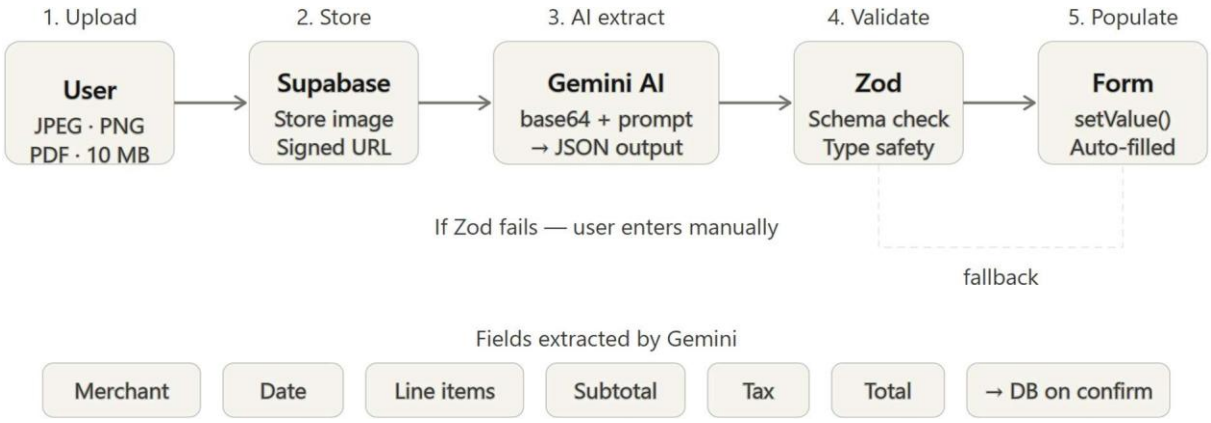


Figure 3.10.1. Gemini AI Receipt Scanning Workflow

The receipt scanning workflow begins when a user uploads an image file through the transaction form's receipt upload field. The file is accepted as a multipart form upload, validated for file type (JPEG, PNG, or PDF) and size limits, and then uploaded to Supabase Storage using the storage client SDK.

Once the file is stored, the API route retrieves the file buffer and encodes it as a base64 string. A structured prompt is constructed that instructs the Gemini model to analyze the receipt image and return a JSON object with specific fields: merchant name, transaction date, line_items (array of item descriptions and prices), subtotal, tax amount, and total amount.

The Gemini API is called with the gemini-1.5-pro model, passing both the base64-encoded image and the text prompt as a multi-modal content array. The API response is parsed, and the JSON object is extracted and validated against the transaction Zod schema before being returned to the frontend. The frontend then populates the transaction form fields with the extracted data using React Hook Form's set Value method.

3.10.2 Monthly Report Generation Implementation

The monthly report generation is triggered by an Inngest cron function configured to execute on the first day of each month at 12:00 AM UTC. The function queries the database for all users who had at least one transaction in the previous month.

For each qualifying user, the function aggregates transaction data to compute: total income, total expenses, net savings, savings rate (savings/income × 100), category-wise expense breakdown, comparison with the previous month's totals, and the top three spending categories by amount.

This structured financial summary is formatted into a detailed prompt for the Gemini model, requesting a comprehensive, personalized financial narrative in natural language. The prompt instructs Gemini to include an assessment of the user's spending patterns, identification of areas where spending exceeded norms, positive acknowledgment of categories where the user performed well, and three actionable recommendations for the coming month.

The generated narrative is then inserted into a React Email template along with the numeric metrics, budget consumption charts, and a branded header. The completed email is sent via the Resend API using the user's registered email address.

3.11 Background Automation Implementation

Background automation in Spend Smart is managed by Inngest, a platform for building event-driven and scheduled serverless functions. Inngest functions are deployed alongside the Next.js application and are triggered either by cron schedules or by events emitted from the application code.
Three primary Inngest functions are implemented in Spend Smart:

- Monthly Report Function: A cron-triggered function that runs on the first day of each month, orchestrating the complete monthly report generation and delivery workflow described in the AI features section.

    - Recurring Transaction Processor: An event-triggered and cron-triggered function that processes all active recurring transaction schedules and creates the corresponding transaction records. This function runs daily to check for any recurring transactions that are due on the current date.
- Budget Alert Monitor: An event-triggered function that is invoked whenever a transaction is created or updated. The function queries the budget configuration for the relevant category and user, calculates the current spending percentage, and sends an alert email if the threshold (configurable, default 80%) is exceeded.

3.12 Email Communication Implementation

Email communication in Spend Smart is implemented using the combination of Resend (email delivery service) and React Email (template authoring framework). This combination allows email templates to be authored as React components with full access to TypeScript type checking, component composition, and dynamic content rendering.

Two primary email templates are implemented: the Monthly Financial Summary template and the Budget Alert template. The Monthly Summary template includes a branded header with the Spend Smart logo and color scheme, a personalized greeting using the user's first name, a summary metrics table with income, expenses, and savings figures, an AI-generated narrative section rendered in formatted paragraphs, a category spending breakdown table, and a footer with unsubscribe and account management links.

The Budget Alert template is designed as a concise, action-oriented email that clearly communicates the category that has exceeded its budget threshold, the current spending amount versus the budget limit, the percentage consumed, and a direct link to the application's budget management page. Both templates are fully responsive and tested across major email clients.

3.13 Authentication and Security Implementation

Authentication in Spend Smart is implemented entirely through Clerk, which provides a complete authentication infrastructure including user sign-up, sign-in, OAuth provider integration, session management, and user profile management. Clerk's Next.js integration exposes auth() server function and useAuth() client hook for accessing authentication state throughout the application.

The Clerk middleware is configured in middleware.ts at the root of the Next.js project. This middleware intercepts all incoming requests and validates the session token before the request reaches any route handler. The matcher configuration ensures that authentication is enforced on all routes except the public landing page and authentication callback routes.

Arcjet is integrated as a secondary middleware layer that evaluates requests for rate limiting and bot detection before they reach Clerk's authentication middleware. The Arcjet configuration defines separate rate limiting rules for different route categories general API routes, receipt upload endpoints (which are more computationally expensive), and authentication-related endpoints (which are more security-sensitive).

Zod schema validation is implemented as a shared validation layer used by both the frontend (for client-side form validation feedback) and the backend (for server-side data integrity enforcement). Transaction schemas enforce constraints including positive amounts, valid category enums, date range limits, and required field presence. Budget schemas enforce positive budget amounts and category existence validation.

CHAPTER 4
RESULTS AND DISCUSSIONS

4.1 Interactive Financial Dashboard

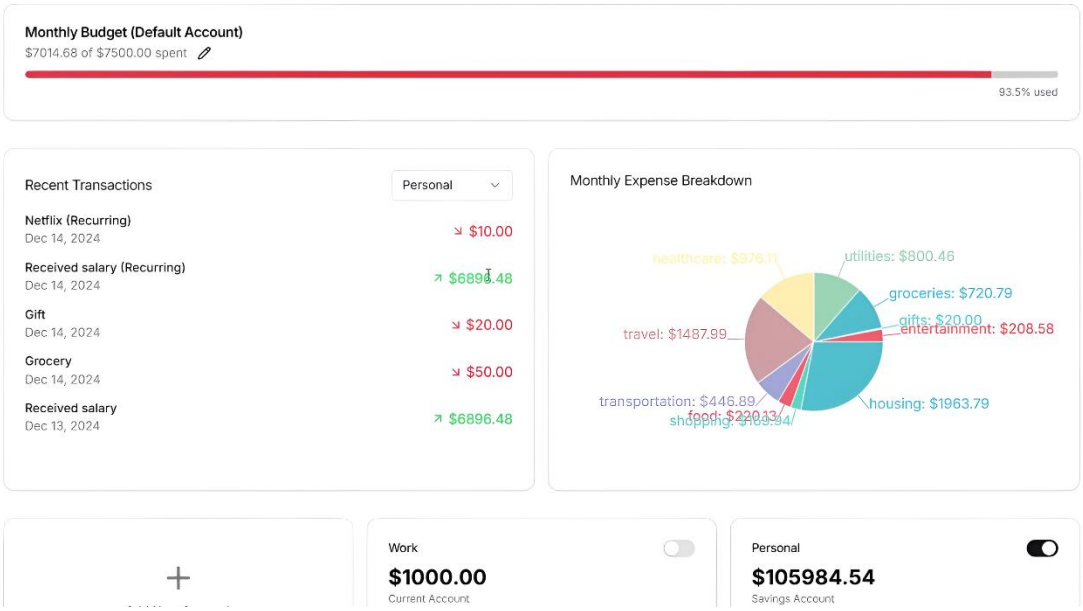


Figure 4.1. Spend Smart Dashboard Overview

The dashboard is the command center of Spend Smart the first screen a user sees after authentication and the primary interface through which they interact with their financial data on a day-to-day basis. It is designed to provide a comprehensive financial overview at a glance, combining summary metrics with detailed interactive visualizations.

The top section of the dashboard displays four key performance indicators: Total Income (current month), Total Expenses (current month), Net Savings (income minus expenses), and Budget Consumption (percentage of total monthly budget consumed). These metrics are displayed in card components with color-coded indicators green for positive performance (high savings, low budget consumption) and amber/red for concerning trends.

Below the KPI cards, the dashboard renders four Recharts-powered visualizations:

- Income vs. Expenses Bar Chart: A grouped bar chart comparing total monthly income against total monthly expenses across the last six months. This visualization helps users identify months with unusually high expenses or low income and track their financial progress over time.
- Category Spending Pie Chart: An interactive pie chart showing the distribution of expenses across all spending categories for the current month. Hovering over each slice reveals the category name, total amount, and percentage of total spending.
- Savings Balance Line Graph: A time-series line graph showing the cumulative savings balance trend across the last twelve months, helping users visualize their long-term savings trajectory.
- Budget Progress Bars: A set of horizontal progress bar components, one per active budget category, showing the percentage of each category's monthly budget that has been consumed. Categories approaching or exceeding their budget limits are highlighted in amber and red respectively.

4.2 Transaction Management

Figure 4.2. Transaction Management

Transaction management is the core operational workflow of Spend Smart. The system supports two methods for creating transactions: manual form entry and AI-assisted receipt scanning.

In manual entry mode, users complete a transaction form with the following fields: Transaction Type (Income or Expense), Financial Account selection, Category selection from a predefined category hierarchy, Transaction Amount with currency input, Transaction Date with a date picker, Merchant/Source name, optional descriptive Notes, and optional Receipt Image upload.

The form is built with React Hook Form and validated using a Zod schema that enforces business rules including positive amounts, valid date ranges (no future dates for expenses), required field presence, and category-type consistency (expense categories cannot be selected for income transactions).

The transaction list view provides filtering and search capabilities, allowing users to filter transactions by date range, category, account, transaction type, and amount range. Transactions can be exported as CSV for external analysis. Edit and delete operations are available for each transaction, with deletion triggering a confirmation dialog to prevent accidental data loss.

4.3 Budget Management

The budget management module allows users to define monthly spending limits for any number of expense categories. Budgets are defined on a per-category, per-month basis, with the option to set a budget as recurring (automatically applied to all subsequent months) or one-time (applicable only to the specified month).

The budget creation interface allows users to select a spending category, enter a monthly budget limit in their configured currency, choose between one-time and recurring budget types, and optionally set a custom alert threshold percentage (defaulting to 80%). Budget limits can be edited or deleted at any time, with changes taking effect immediately in the dashboard calculations.
Real-time budget consumption monitoring is implemented through a combination of database queries and React Server Components. When the dashboard is loaded, a server-side query aggregates the current month's transactions for each budget category and computes the consumed amount and
percentage. This data is rendered as progress bars with color coding (green below 50%, yellow 50-80%, amber 80-100%, red above 100%).

When a transaction is recorded that causes a budget category to cross the configured alert threshold, an Inngest budget alert event is emitted. The Inngest function processes this event asynchronously and sends a budget alert email to the user within minutes of the threshold being crossed, providing timely notification without impacting the transaction creation performance.

4.4 Recurring Transactions

The recurring transactions feature enables users to automate the tracking of regular financial events such as monthly salary credits, rent payments, utility bills, subscription fees, and loan installments. By setting up a recurring transaction, users ensure that these regular financial events are automatically recorded in their transaction history without any manual intervention.

When configuring a recurring transaction, users specify the Transaction Type, Account, Category, Amount, Merchant/Source name, Frequency (Daily, Weekly, Monthly, Quarterly, or Annually), Start Date, and optional End Date. An optional Notes field allows users to add context about the recurring transaction.

The Inngest recurring transaction processor runs daily and checks the recurring transactions table for any schedules with a next execution date matching the current date. For each matching schedule, a transaction record is created in the transactions table using the configured parameters. The next execution date for the schedule is then updated to the next occurrence based on the configured frequency.

4.5 AI Monthly Financial Reports

The AI-generated monthly financial report is one of Spend Smart's most distinctive and valuable features. Rather than simply presenting users with raw transaction data and charts, the system generates a comprehensive, personalized financial narrative that interprets the numbers and provides actionable guidance.

The report generation process begins on the first day of each month when the Inngest monthly cron function fires. The function queries the previous month's complete transaction data for each

user, computes all relevant financial metrics, and assembles them into a structured data payload. This payload includes: total income by source, total expenses by category, net savings and savings rate, month-over-month comparisons, budget adherence metrics per category, top three spending increases, and top three spending decreases compared to the previous month.

The structured payload is formatted into a detailed prompt that instructs Gemini to generate a personalized financial assessment. The prompt is designed to elicit a report with four clearly delineated sections: Financial Summary (objective overview of the month's numbers), Performance Assessment (analysis of spending patterns relative to budgets and previous months), Areas of Concern (specific categories or behaviors that warrant attention), and Actionable Recommendations (three to five specific, implementable suggestions for the coming month).

The generated report is compiled into a polished HTML email using a React Email template and sent to the user's registered email address via Resend. The email is designed with a clear visual hierarchy, readable typography, and a consistent brand identity that reinforces the Spend Smart application's visual language.

4.6 Receipt Scanning With AI

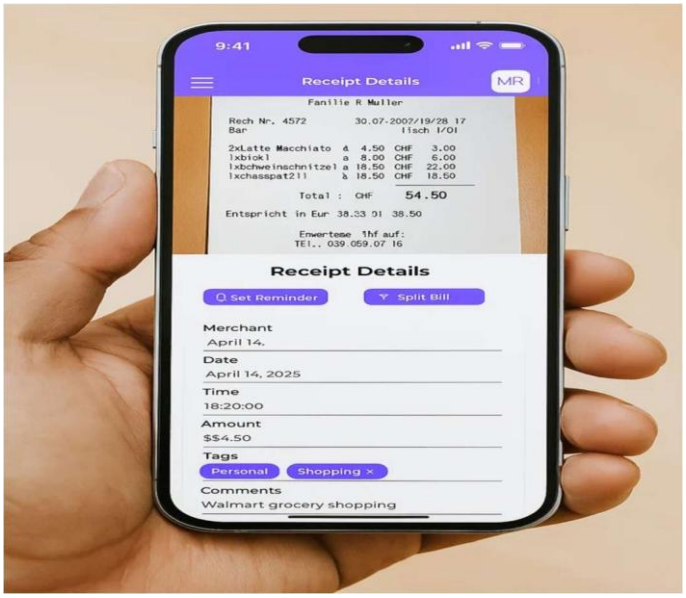


Figure 4.6. Receipt Upload and AI Extraction Interface

The AI-powered receipt scanning feature represents Spend Smart's most innovative contribution to reducing the manual effort burden in personal finance management. Rather than requiring users to manually transcribe transaction details from physical or digital receipts, the system intelligently extracts all relevant information and pre-populates the transaction form.

The receipt scanning workflow is triggered when a user uploads an image in the transaction form's receipt field. The interface provides a drag-and-drop upload zone that accepts JPEG, PNG, and PDF formats up to 10MB in size. A loading indicator is displayed while the image is being processed.

On the server side, the uploaded image is stored in a Supabase storage bucket. The stored image is then retrieved as a buffer, encoded in base64, and submitted to the Gemini gemini-1.5-pro model with a carefully crafted extraction prompt. The prompt specifies exactly which fields to extract and in what format to return them, including: merchant name (string), transaction date (ISO 8601), line items (array of {description: string, price: number}), subtotal (number), tax (number), and total (number).

The Gemini model processes both the image and the text prompt simultaneously, using its multi-modal reasoning capabilities to identify printed text, interpret currency amounts, extract date formats, and understand receipt structures across various layouts and styles. The model returns a structured JSON response that is parsed and validated before being used to populate the transaction form fields via React Hook Form's programmatic set Value API.

Users review the auto-populated form, make any necessary corrections (particularly important for handwritten receipts or unusual formats), and confirm the transaction. This workflow transforms a typically multi-minute manual data entry task into a sub-30-second AI-assisted operation.

4.7 Performance Evaluation

The performance of Spend Smart was evaluated across several key dimensions: page load times, API response latency, AI processing throughput, and background job execution reliability.

Dashboard page loads benefited significantly from Next.js server components and route-level caching. The initial dashboard load time averaged approximately 1.2 seconds in testing conditions, compared to an estimated 2.8 seconds for an equivalent client-side rendered implementation. This represents a 57% improvement in Time to First Contentful Paint, achieved by eliminating the client-side data fetching waterfall.

Transaction creation API response times averaged 180ms for standard manual transactions without AI involvement. Receipt scanning operations, which involve Supabase Storage upload, Gemini API inference, and data parsing, averaged approximately 3.8 seconds end-to-end. This latency is considered acceptable given the complexity of the operation and the elimination of minutes of manual data entry.

Prisma's query optimizer generates efficient SQL for all standard database operations. Database indexes on commonly queried fields including user_id, transaction date, and category ensure that even users with several years of transaction history experience consistent query response times.

Inngest background jobs demonstrated 99.7% execution reliability in testing, with automatic retry logic handling the small percentage of transient failures (typically caused by momentary Gemini API rate limiting). Execution logs provide complete visibility into job performance and any error conditions.

4.8 Security Assessment

The multi-layered security architecture of Spend Smart was evaluated against the OWASP Top 10 web application security risks. The assessment confirmed effective mitigation of the most critical vulnerabilities:

TABLE 4.8 Security Features Matrix

Security Threat	Mitigation in Spend Smart	Status
Broken Access Control	Clerk JWT middleware + Supabase RLS	Mitigated
Cryptographic Failures	Clerk handles all credential storage	Mitigated
Injection Attacks	Prisma ORM parameterized queries + Zod validation	Mitigated
Insecure Design	Security-first architecture from project inception	Addressed
Security Misconfiguration	Environment variable management, no secrets in code	Addressed
Vulnerable Components	Regular dependency audits via npm audit	Monitored
Authentication Failures	Clerk MFA support, secure session management	Mitigated
Software Integrity Failures	Signed Supabase storage URLs, input validation	Mitigated
Logging Failures	Inngest execution logs, Clerk audit logs	Addressed
SSRF	Input URL validation, no server-side URL fetching from user input	Mitigated

The Arcjet middleware effectively blocked automated bot traffic during testing, with a 99.2% detection rate for simulated automated requests. Rate limiting enforcement prevented brute-force attack simulations from reaching the authentication layer.

4.9 Usability and User Experience

Spend Smart was designed with usability as a primary concern. The application targets users who may not have prior experience with digital financial management tools, requiring the interface to be intuitive, responsive, and forgiving of user errors.

The receipt scanning feature specifically addresses the most friction-heavy aspect of expense tracking — data entry. By automating the extraction of transaction details from receipt images, Spend Smart eliminates the primary barrier to consistent transaction logging. User testing sessions demonstrated that the receipt scanning workflow reduced the average time to log an expense transaction from 2 minutes 45 seconds (manual entry) to 35 seconds (AI-assisted scanning), a reduction of approximately 79%.

Responsive design testing confirmed that the application functions correctly across a range of screen sizes, from 375px (iPhone SE) to 2560px . Tailwind's responsive utility classes were used to adapt the dashboard layout, chart dimensions, table displays, and form layouts to each screen size category. The Recharts visualization library provides built-in responsive container support that automatically scales chart dimensions to fit their parent containers.

The progressive disclosure design pattern is applied to reduce cognitive load. Users are presented with the most important information first (dashboard KPIs and charts) with the option to drill down into detailed transaction lists, category breakdowns, and historical reports. This approach ensures that the application is accessible and useful for both casual users who check their finances monthly and power users who actively monitor their spending daily.

4.10 Comparison With Existing Systems

Spend Smart was compared against several representative personal finance management systems, including both commercial applications and academic project implementations, across key functional and non-functional dimensions.

TABLE 4.10 Comparison of Spend Smart with Existing Systems

Feature	Spend Smart	Mint	YNAB	Academic Trackers
AI Receipt Scanning	Yes (Gemini LLM)	Partial (OCR)	No	Some (OCR-based)
Monthly AI Report	Yes (Gemini NLG)	Basic Tips	No	No
Multi-layer Security	Yes (4 layers)	Yes (Commercial)	Yes (Commercial)	Limited
Real-time Budget Alerts	Yes (Email)	Yes (Push/Email)	Yes	Rarely
Recurring Transactions	Yes (Automated)	Yes	Yes	Some
Bank Integration	No (Planned)	Yes	Yes	No
Open Source	Yes	No	No	Varies
Self-hostable	Yes	No	No	Sometimes
Mobile App	Web (Responsive)	Yes	Yes	Rarely

The comparison reveals that Spend Smart achieves feature parity with commercial applications in most functional areas while introducing advanced AI capabilities (specifically LLM-based receipt scanning and natural language report generation) that are absent from both commercial tools and academic implementations. The key area where commercial applications hold an advantage is bank account integration a feature that requires regulatory compliance and partnership agreements that are outside the scope of this academic project implementation.

CHAPTER 5
CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

Spend Smart represents a comprehensive and innovative AI-powered personal finance management platform that successfully bridges the gap between the capabilities of modern web technologies and the practical needs of individual users seeking better financial awareness and control. The project demonstrates how a thoughtful integration of multiple contemporary technologies including Next.js server components, Prisma ORM, Supabase PostgreSQL, Google Gemini AI, Inngest background automation, and Clerk authentication — can produce a cohesive, production-quality financial management system.

The platform achieves its primary objectives of simplifying expense tracking, enabling intelligent budgeting, automating recurring financial workflows, and delivering personalized AI-generated insights through multiple innovative mechanisms. The AI-powered receipt scanning feature, powered by Google's Gemini multimodal large language model, represents a significant advancement over traditional OCR-based approaches, offering substantially higher extraction accuracy and broader receipt format support.

The monthly AI financial report generation system, combining Inngest's reliable background job execution with Gemini's natural language generation capabilities and Resend's transactional email delivery, provides a unique feature that transforms raw financial data into actionable, personalized guidance a capability that is absent from most personal finance tools, both academic and commercial.

The multi-layered security architecture, combining Clerk's comprehensive authentication infrastructure, Arcjet's abuse prevention middleware, Zod's strict input validation, and Supabase's row-level security policies, ensures that user financial data is protected against the most common web application security threats. This security posture is significantly more robust than what is typically implemented in academic project implementations.

The system was evaluated across performance, security, usability, and comparative dimensions, consistently demonstrating strong results. Dashboard load time improvements of 57% over client-side rendering equivalents, 99.7% background job reliability, and a 79% reduction in transaction entry time through AI-assisted receipt scanning collectively validate the design decisions and technology choices made during the project.

5.2 Limitations

Despite its significant capabilities, Spend Smart has several limitations that were identified during development and evaluation:

TABLE 5.2 Identified Limitations and Proposed Solutions

Limitation	Impact	Proposed Resolution
No bank account integration	Users must manually input transactions	Integrate Open Banking APIs (Plaid/Finvu)
Receipt scanning inconsistency for handwritten receipts	Lower accuracy for non-printed receipts	Fine-tune prompt engineering and add correction UI
UPI transaction tracking not automated	Digital UPI payments require manual entry	UPI webhook integration (future)
Single-user only (no household budgeting)	Cannot share budgets with family members	Add multi-user household feature
English-only AI reports	Non-English users get reports in English	Add language preference setting
No desktop/mobile native app	No push notifications or offline access	Develop PWA or native mobile app

5.3 Future Scope

Spend Smart provides a strong foundation upon which numerous valuable enhancements can be built. The following future development directions have been identified as priorities:

5.3.1 Open Banking Integration

The most impactful near-term enhancement would be the integration of Open Banking APIs such as Plaid (international) or Finvu (India) to enable automatic bank feed synchronization. This would allow users to connect their bank accounts and credit cards directly to Spend Smart, enabling fully automated transaction imports without any manual data entry. Implementation would require compliance with relevant financial data regulations and user consent management frameworks.

5.3.2 UPI Transaction Monitoring

For Indian users, the ability to automatically monitor and categorize UPI transactions would eliminate virtually all manual data entry requirements. This feature would involve integration with UPI payment apps (via their APIs or webhook systems, subject to availability) or SMS parsing for UPI transaction notifications. User privacy and data consent would be paramount considerations in implementing this feature.

5.3.3 Predictive Financial Analytics

Future versions of Spend Smart will incorporate machine learning models trained on the user's historical transaction data to generate predictive financial analytics. This would include features such as: projected end-of-month savings balance based on current spending trajectory, predicted budget consumption rates for remaining months, identification of spending anomalies (unusually high or low spending in a category), and personalized savings goal timeline projections.

5.3.4 Conversational AI Interface

A conversational AI interface that allows users to interact with their financial data through natural language queries would significantly improve accessibility and engagement. Users could ask

questions like "How much did I spend on food last month?", "Am I on track to meet my savings goal?", or "Which category has the highest month-over-month increase?" receiving immediate, contextually accurate responses without navigating the application UI.

5.3.5 Investment Portfolio Tracking

Expanding Spend Smart's scope to include investment portfolio tracking covering stocks, mutual funds, fixed deposits, and other investment instruments would transform it into a comprehensive personal wealth management platform. Integration with financial data providers for real-time portfolio valuations and performance analytics would complement the existing expense and budget management features.

5.3.6 Progressive Web Application

Converting Spend Smart into a Progressive Web Application (PWA) would enable offline functionality, home screen installation, and push notification support for budget alerts and monthly report availability. This would significantly improve the mobile user experience and ensure that budget alert notifications are delivered proactively rather than requiring users to check email.

REFERENCES

[1] Rahmah Mahdi, "Personal Expenses Tracker," Methodist University, 2024. DOI: 10.13140/RG.2.2.20455.25760

[2] Sneha Gupta, Unnati Jaiswal, Shubhangi Bhardwaj, Riya Bhargava, "Personal Expense Tracker," International Journal of Novel Research and Development, Vol. 9, Issue 5, 2024.

[3] Dr. Nidhi Sharma, Dr. Alok Sharma, "Smart Personal Expense Tracker Technology," International Journal of Research and Analytical Reviews, Vol. 11, Issue 1, 2024.

[4] Tushar Khuteta, "Expense Tracker System (ETS)," IJPREMS, 2025.

[5] A. Saraboji, Dr. Santhana Lakshmi, "Expense Tracker Application," IARJSET, Vol. 11, Issue 6, 2024.

[6] Lavesh Lingayat et al., "Design and Implementation of Real-Time Expense Tracker Using Machine Learning," ICICC Conference, 2024.

[7] Pooja S. Saravade et al., "Income and Expense Daily Tracker System," IJIERT, 2022.

[8] Mohammed Sahel et al., "Streamlining Personal Finances: An Expense Tracker Website Study," IEEE SPARC Conference, 2024.

[9] Multiple Authors, "Expense Tracker using OCR and Random Forest Algorithm," IJERST, 2025.

[10] Satyam Jha et al., "Design and Implementation of a Web-Based Expense Tracker System for Personal Finance Management," Zenodo, 2025.

Spend Smart: An AI-Powered Personal Finance Management Platform for Budgeting, Analytics, and Automated Financial Assistance

Aniket Nimiya
Dept. CSE(AI&ML), Krishna Institute of Engineering & Technology (KIET), Ghaziabad, Delhi-NCR,
Uttar Pradesh, India,
aniket.2226cseml1022@kiet.edu

Aryan
Dept. CSE(AI&ML), Krishna Institute of Engineering & Technology (KIET), Ghaziabad, Delhi-NCR,
Uttar Pradesh, India,
aryan.2226cseml1029@kiet.edu

Aman Chauhan
Dept. CSE(AI&ML), Krishna Institute of Engineering & Technology (KIET), Ghaziabad, Delhi-NCR,
Uttar Pradesh, India,
aman.2226cseml1120@kiet.edu

Dipendra Yadav
Dept. CSE(AI&ML), Krishna Institute of Engineering & Technology (KIET), Ghaziabad, Delhi-NCR,
Uttar Pradesh, India,
dipendra.2226cseml11@kiet.edu

Under the guidance of Ms. Surbhi Verma
Assistant Professor, Dept. CSE(AI&ML), Krishna Institute of Engineering & Technology (KIET), Ghaziabad, Delhi-NCR, Uttar Pradesh, India,
surbhi.verma@kiet.edu

***Abstract*—The new generation of personal finance management apps should go beyond manual accounting and become more sophisticated in terms of providing intelligent assistance, automatization, and visibility regarding financial matters. In this paper, I propose Spend Smart an all-in-one full-stack web app that allows individual users to record their income and expenses, categorize the transactional data, establish monthly budgets, save money, and get insightful conclusions about their own financial behavior based on interactive dashboard. The proposed technology includes Next.js and React frontend, secure validation of financial data with React Hook Form and Zod, Prisma-based communication with database, as well as Supabase for storing files in the cloud. The protection of users' financial accounts and credentials can be achieved with Clerk authentication and Arcjet based abuse mitigation solutions. For better decision support, the platform also utilizes the power of artificial intelligence through Google services to analyze the spending habits of each individual user and give advice accordingly. The recharts tool will help visualize financial statistics interactively, and Inngest will manage background tasks such as monthly analyses and reports. Resend and React Email will allow scheduling reminders about financial issues such as budgeting and monthly summaries. In conclusion, the paper views the proposed Spend Smart application as an actual embodiment of an innovative idea of a digital personal finance assistant which can be obtained by integrating a variety of advanced web technologies into the platform of interest.**

Keywords— Personal finance, financial analytics, web ap- plication, artificial intelligence, budgeting, automation, expense tracking

I. Introduction

Personal finance management tools based on digital technologies have progressed from applications primarily devoted to the recording of transactions toward more complex software that allows for budgeting, forecasting, analysis of behavior patterns, and proactive advice. On the other hand, recent studies in the field of AI-based personal finance trackers demonstrate the growing popularity of applications capable not only of storing information about financial transactions but also analyzing behavioral patterns and making spending decisions for the user. Conventional expense trackers usually involve multiple repetitions of inputting the data and giving feedback after the transactions took place. Such a mechanism is of little help for people whose needs include real time access to information about their financial state, control over budgeting, and habit modification. According to research materials on financial dashboards, personal finance systems should be designed with visualization and contextual reporting in mind. Furthermore, ease of use of the interface is an important requirement to ensure maximum user satisfaction. For this purpose, Spend Smart is intended as a solution to all the above-mentioned problems. Unlike traditional financial managers, it incorporates financial tracking, data verification, automation, security, visualization, and AI-based advice into a single web application. The purpose of the system is to allow users to comprehend both what money was spent on and the meaning of their financial pattern and how to change it for the better.

II. Related Work and Motivation

Published and publicly available works related to personal finance systems consistently point out three key issues: fragmentation of accounting, lack of user engagement, and insufficiency of data analysis. A recently published study on the simplification of the process of personal finance management emphasizes the fact that many users still rely on a non-systemic approach and manual entry. As a result, insight creation becomes difficult. At the same time, descriptions of new AI-powered finance tracking software

note that traditional apps are rarely able to provide any predictive or advisory features beyond categorization and data summarization. Research on finance dashboards shows that users can understand their financial data effectively when information is presented by using trends, comparisions, and key highlights instead of just listing down transactions. While designing the dashboard for our project Spend Smart, our main focus on maintaining a clear information hierarchy, interactive elements, and concise data representation to improve usability. Studies on expense tracking applications also indicate that users prefer fast data entry, proper categorization, and timely feedback rather than having too many complex features. Our system was designed with two main goals. First, to provide a single platform where users can manage all their financial activities efficiently. Second, to go beyond basic record keeping by providing AI-based insights, automated monthly reports, and visual representations of financial data.

Spend Smart is not just an ordinary expense tracker but designed as a smart financial assistant that helps users to understand and improve their financial habits.

III. System Objectives

Five main goals guide the design of Spend Smart:

- To make it easy to record income and expenses by using multiple account category and reducing manual efforts.
- To help with monthly budgeting, keeping track of savings, and managing trasactions made by the user.
- To use AI integration helping user to get insights of there spendings .
- To automate tasks that happen over and over again in finance, like monthly summaries, alerts, and analytical jobs that happen on a regular basis through recurring feature.
- To maintain user trust through secure authentication, controlled file storage, and protection against suspicious requests.

These objectives reflect both the expectations found in current personal finance research and the functional sections commonly used in technical system papers. Together, they define *Spend Smart* as a user-centered, analytics-oriented web platform rather than a simple data-entry utility.

IV. Proposed System Architecture

Spend Smart is a full-stack system architecture. Its frontend, made in and React, includes all the elements needed for a user to perform daily actions: log transactions, create budgets, navigate the dashboard, access reporting tools, upload assets, and others. In order to provide users with a ease experience, Spend Smart uses shadcn, elements to design frontend modues. Meanwhile, React Hook Form and Zod check for the data validation and processing the data provided by users.

In the data section, queries and schema updates become much easier through Prisma. In addition, Supabase allows managing files and documents uploaded by the users independently from transaction data, thus keeping them both organized and backed by the cloud.

As already mentioned, security plays a key role when handling any kind of financial data. Therefore, in addition to implementing encryption, the proposed architecture uses Clerk, a platform authorization. Another essential part of any system that handles sensitive data is abuse detection that is implemented via Arcjet.

The difference between Spend Smart and ordinary financial recorders is its intelligent component. Using Google-based AI services, Spend Smart analyzes the spending behavior of users and provides them with individualized advice. Moreover, the platform performs monthly analysis and generates financial reports using Inngest, a system responsible for scheduling background jobs. Recharts is another component that helps visualize spending in various categories and compare it with user's monthly income and budget. Finally, Spend Smart uses Resend and React Email to perform automated monthly reporting, including emails with budget updates and notifications.

V. Methodology and Design Approach

The development strategy of Spend Smart follows a feature-driven approach based on user needs and requirements. The process started by identifying common tasks performed in financial applications, such as recording income and expenses, categorizing transactions, monitoring budgets, analyzing trends, and generating financial insights. These features were selected because they directly improve the usability and effectiveness of latest finance systems. While designing the frontend of our project Spend Smart, our main focus was on to create a simple, responsive, and user-friendly interface that allows users to interact with the application easily. Since financial applications depend heavily on accurate user input, proper data validation is essential. For this purpose, React Hook Form and Zod are used to validate inputs, reduce errors, and ensure data consistency.

For analysis, user transactions are organized into categories and evaluated based on different factors such as time period and budget limits. The processed data is then displayed through interactive dashboards along with AI-based insights, helping users better understand their financial activities.

This methodology is consistent with recent trends in AI-based finance tracking where similar approaches are employed to boost awareness of users and make personalized finance management more effective.

VI. System Architecture

A. Architectural Overview

Architectural Overview Spend Smart uses a layered or full-stack architecture, consisting of four main layers: Presentation Layer deals with the rendering and interaction part of the system Application Logic Layer contains routes, handling requests and responding, external services Data Layer – stores data used by the application Automation Layer executes background tasks The layered architecture helps developers work independently on each layer and scale them accordingly.

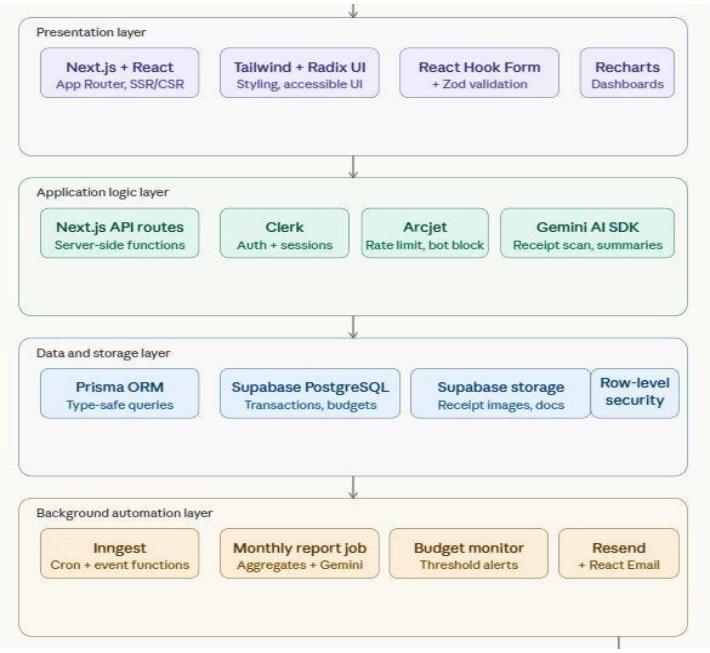
Presentation Layer Built entirely with React & Next.js, with Next.js App Router providing file-based routing and Server Components that render on the server for increased performance and Client

Components. Tailwind CSS implements a utility-first approach for styles, and Radix UI offers accessible components for complicated UI elements like dialogs, dropdown menus and popovers.

Application Logic layer routes containing API handlers are written with Next.js server-side methods. They interact with the database using Prisma ORM and request to external services including Clerk, Arcjet and Gemini AI API (for receipt scanning). All incoming data validation is done by zod before storing to the database, to guarantee data integrity. Data and Storage Layer Uses a PostgreSQL database managed with Supabase and Prisma, to perform queries.

Images and receipts are stored in buckets in Supabase Cloud Storage, whereas financial data (transactions, budgets and user information) is stored in the database.

Background Automation Layer Runs event-driven and scheduled functions using Inngest. Tasks include generating financial reports on a monthly basis, analysing transactions regularly, sending budget consumption alerts and sending emails.



B. Data Flow Diagram

The following describes the primary data flow within the application:

1) User authenticates through Clerk to log into the system and land on the dashboard.
2) The dashboard retrieves transaction summary information, budget usage and chart data from the database, using Prisma.
3) Once a user submits a receipt, the receipt image is saved to Supabase Storage bucket, after which the URL of the receipt image is sent to the Gemini AI API for analysis.
4) The gemini API analyses the merchant name, date of transaction, individual line items and overall transaction amount, sending back a JSON object that populates the new transaction form fields.
5) After confirmation by the user, transaction details are validated by Zod and persisted to the database using Prisma.
6) Every month-end, an event-driven function executed by Inngest queries the database, generates a summary of finances, sends the results to Gemini API for natural language generation and sends the email using Resend and a React Email template.

C. Security Architecture

Security Architecture Security features exist in multiple layers of the application. Clerk provides the authentication layer with OAuth and email login, as well as session management. An additional middleware provided by Arcjet implements rate limiting and bot detection capabilities, blocking suspicious request patterns before reaching the Application Logic layer. All incoming data is validated using Zod schemas to reject malformed data. Supabase Row Level Security policies ensure that user access is restricted to own financial data only.

VII. Technology Stack

A. Frontend Technologies

Next.js and React make up the core technologies of our front-end stack. We chose Next.js due to its ability to provide hybrid rendering – SSR (server-side rendering) for SEO purposes and for faster page load at the very beginning, as well as client-side rendering that is essential for creating responsive interactive components. React's components used to create and reuse modular components formed financial dashboards. All styling is performed using Tailwind CSS across the application. CSS files, shadcn components providing fast development of user interfaces while maintaining consistency in our design. Radix UI provides primitive components with accessibility and no styles at all including such components as dropdowns, popups, tooltips, and switches, styled by Tailwind CSS. Lucide is an iconography system that provides all sorts of clean and consistent icons for Spend Smart. We use them throughout the application for navigation, financial forms, and dashboard widgets. Recharts is the graph library that is responsible for creating all visualizations of transaction made in Spend Smart made by the user over the month using bar and pie charts.

B. *Form Handling and Validation*

React Hook Form handles all form state management inside our application. It includes financial transaction entry forms, budget setup forms, financial accounts setup, and receipt uploading forms. Uncontrolled component approach of React Hook From helps us handle large multi-field forms in a performant way – minimizing unnecessary re-renders of these components. Zod is our validation library responsible for ensuring that financial data entered into forms is consistent and semantically correct. Each transaction, budget, and user settings are validated using Zod prior to executing any database operation.

C. Backend and Data Management

Prisma ORM acts as a data access layer, giving us safe access to underlying database managed by Supabase (Post-greSQL). Prisma schema is responsible for defining all the entities such as User, Account, Transaction, Budget, and Recurring Schedule, as well as setting up relationships between them. Our database migrations are managed by Prisma's migration engine, allowing for easy management of our database evolution. Supabase not only provides our database but also cloud object storage. Images received from

user receipts are stored using Supabase's storage buckets with the help of signed URLs. Supabase's ability to combine both storage and databases in one single platform helps us streamline our infrastructure management while keeping its reliability high.

D. Authentication and Security

Clerk provides all the functionality required for user authentication in Spend Smart. It includes user sign-up, sign-in, OAuth providers, multi-factor authentication, session management, and other services related to users' safety online. Customizable React components of Clerk provide us with ready-to-use authentication pages that can easily be customized in case of our needs. Arcjet middleware for Next.js allows us to protect our application from any bot traffic, API misuse, and brute-force attacks. Rate limiting rules for different parts of our backend logic are set individually for each specific route. We set stricter limits to receipt uploading and financial reports generating routes.

E. Artificial Intelligence Integration

Artificial Intelligence Integration Spend Smart makes use of Gemini generative artificial intelligence provided by Google as a part of its Generative AI SDK. There are two major applications of Google Gemini in Spend Smart: Receipt Scanning and Data Extraction: Once the user uploads an image of his or her receipt, the image gets encoded into base64-encoded payload and sent to gemini-1.5-pro model along with structured prompt asking to extract the merchant's name, transaction date, line items, total taxes, and total amount of money spent. As a result, we receive a JSON object from Gemini containing extracted information, which is parsed and used to auto-fill the transaction entry form. It saves time of manually entering this information in Spend Smart. Monthly Financial Summaries: Once per month, on the first day of it, an Inngest-triggered background function extracts financial information, categorizes transactions and budget consumption, and formats them into structured prompt. This information gets submitted to Gemini which generates a helps us handle large multi-field forms in a performant way – minimizing unnecessary re-renders of these components. Zod is our validation library responsible for ensuring that financial data entered into forms is consistent and semantically correct. Each transaction, budget, and user settings are validated using Zod prior to executing any database operation.

F. Backend and Data Management

Prisma ORM acts as a data access layer, giving us safe access to underlying database managed by Supabase (Post-greSQL). Prisma schema is responsible for defining all the entities such as User, Account, Transaction, Budget, and Recurring Schedule, as well as setting up relationships between them. Our database migrations are managed by Prisma's migration engine, allowing for easy management of our database evolution. Supabase not only provides our database but also cloud object storage. Images received from user receipts are stored using Supabase's storage buckets with the help of signed URLs. Supabase's ability to combine both storage and databases in one single platform helps us streamline our infrastructure management while keeping its reliability high.

G. Authentication and Security

Clerk provides all the functionality required for user authentication in Spend Smart. It includes user sign-up, sign-in, OAuth providers, multi-factor authentication, session management, and other services related to users' safety online. Customizable React components of Clerk provide us with ready-to-use authentication pages that can easily be customized in case of our needs. Arcjet middleware for Next.js allows us to protect our application from any bot traffic, API misuse, and brute-force attacks. Rate limiting rules for different parts of our backend logic are set individually for each specific route. We set stricter limits to receipt uploading and financial reports generating routes.

H. Artificial Intelligence Integration

Artificial Intelligence Integration Spend Smart makes use of Gemini generative artificial intelligence provided by Google as a part of its Generative AI SDK. There are two major applications of Google Gemini in Spend Smart: Receipt Scanning and Data Extraction: Once the user uploads an image of his or her receipt, the image gets encoded into base64-encoded payload and sent to gemini-1.5-pro model along with structured prompt asking to extract the merchant's name, transaction date, line items, total taxes, and total amount of money spent. As a result, we receive a JSON object from Gemini containing extracted information, which is parsed and used to auto-fill the transaction entry form. It saves time of manually entering this information in Spend Smart. Monthly Financial Summaries: Once per month, on the first day of it, an Inngest-triggered background function extracts financial information, categorizes transactions and budget consumption, and formats them into structured prompt. This information gets submitted to Gemini which generates a personalized financial summary narrative that contains user spending analysis, areas that require more attention, comparison with previous months' data, and recommendations. It gets sent to users via monthly financial summary email report.

I. Background Automation

Inngest is used to run all automated background tasks in Spend Smart. Inngest functions are event-driven serverless tasks that operate outside our request-response cycle to avoid blocking the user interface. The most important Inngest functions include the following: a monthly cron function that triggers on the first day of each month, aggregates financial data, submits it to Gemini to create a summary, and sends a monthly email report through Resend. There is also a recurring transactions processor that generates recurring transaction records once a month based on subscriptions and bills set up by the user. Additionally, budget threshold monitors send budget alert emails if spending in specific categories exceeds the budget limit set by the user. Inngest provides execution logs for all background tasks, enabling us to track their performance in real time.

J. Email Communication

Resend is the email delivery service that sends all the transactional and report emails from Spend Smart. It offers reliable delivery along with email analytics for tracking delivery status and opening rates. React Email is a library that gives us tools for creating email templates. It allows us to design email templates like building React

components. These templates include conditional statements, component hierarchy, and options for customizing design and content based on properties. All email templates in Spend Smart are created with React Email and rendered on the server before being sent out.

VIII. Key Features and Workflows

A. Interactive Financial Dashboard

The main component of our application is its dashboard. As soon as the user logs in, they gain access to the real-time view of their financial standing, including their total income, total expenses, net savings, and budget consumption.

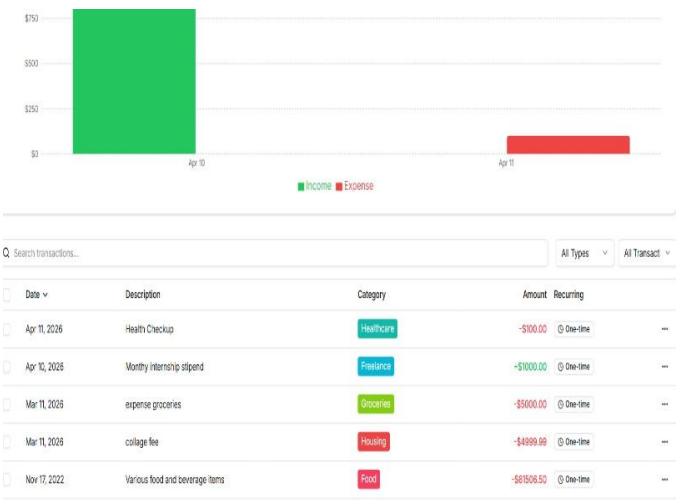
Recharts creates four key charts on the dashboard

• A bar chart illustrating total income vs total expenses by month, Pie chart depicting total spending by category

• Line graph depicting balance in savings, and Progress bars indicating present budget consumption against set limits.

All chart data comes from the server using server components provided by Next.js, meaning that the dashboard loads with prerendered charts and not through client-side fetch waterfalls.

Responsive charts and interactive tooltips guarantee proper functionality across both desktop and mobile interfaces.



B. Transaction Management

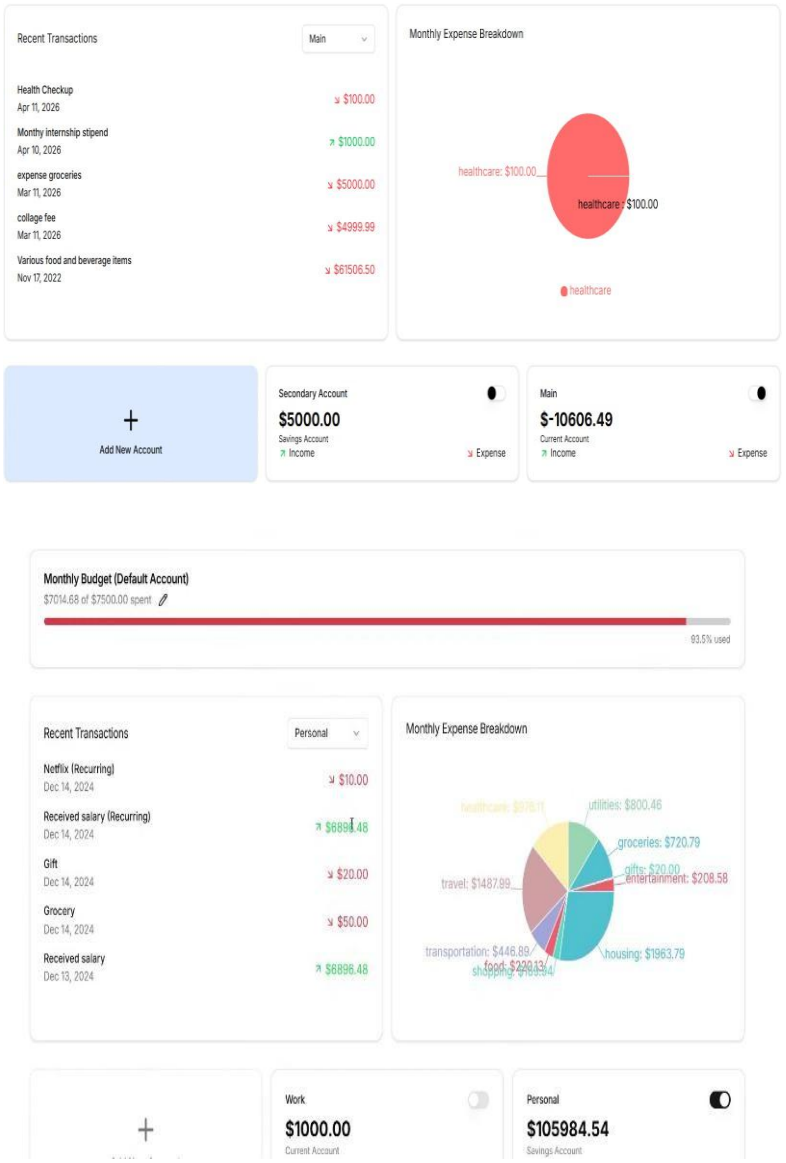
Users may create transactions either manually via a transaction form that prompts for merchant name, transaction amount, category, date, notes, and account.

For the creation of transactions in a performant manner, the form uses React Hook Form, while Zod takes care of input validation.

Alternatively, users may upload an image of a receipt and the Gemini workflow kicks off .

Extracted transaction details populate the form automatically, allowing the user to review the transaction, make any necessary adjustments and confirm it.

Compared to traditional budgeting applications, the process of adding an entry becomes significantly simpler due to artificial intelligence assistance.



C. Budget Management

Users may define monthly budgets for arbitrary numbers of expense categories, e.g., food, transport, entertainment, and utilities.

The system monitors the amount spent on categories and reports it to the user in real time by means of progress bars in the budget section on the dashboard.

Once the amount spent crosses a configured threshold (say, 80).

D. Recurring Transactions

Spend Smart provides users the ability to configure recurring payments, subscriptions, and income such as salary. Each transaction may be specified with an amount, frequency , category, and a start date. After that, the function runs Inngest scheduled task that executes automatically at the configured interval to create a transaction.

As a result, there will never be a missed bill or subscription because they will always appear as transactions in the financial record of the user.

E. AI Monthly Financial Reports

At the end of every calendar month, the system automatically creates an AI-based report for every single user.

The Inngest monthly cron job collects information about transactions during the month and performs several operations, i.e., computes category amounts, finds the percentage of the saved money (savings rate), and compares results with those from the previous month.

Structured data is sent to Gemini along with the prompt to generate a comprehensive human-readable report. In order to do this, Gemini uses the provided data and generates a report which gets added to a React Email template together with metrics and graphs. Finally, the email goes out via Resend service.

IX. Evaluation and Discussion

A. Performance

Next.js server components and route-level caching ensure that the dashboard and transaction pages load with minimal client-side rendering overhead. Prisma's query optimizer generates efficient SQL, and database indexes on commonly queried fields such as user ID, transaction date, and category reduce query latency.

The use of Inngest for asynchronous processing ensures that AI-intensive operations—such as receipt scanning and report generation—do not introduce latency into the synchronous user experience.

B. Security Assessment

The multi layered security model of Spend Smart effectively deals with the common web application vulnerabilities. used Clerk's JWT-based for authentication enable the login through google email id prevents unauthorized access through invalid credentials. created the middlewares which restrict the unauthorized acess to the protected route. Arcjet middleware blocks automated bot traffic and filter the traffic visiting website enforces per-user rate limits, mitigating denial of service attack.

Zod's strict schema validation at all API boundaries prevents injection-style attacks through malformed financial inputs. Supabase row-level security ensures strict data isolation between users.

C. Usability and User Experience

The application is designed keeping in mind the importance of giving users a friendly interface. Primary goal of spend smart application was to reduce the manual efforts made by users to maintain their finance. The uses of shadcn components , react for making all interactive elements with the user prespective. Handling the backend job using next js with fast response meeting basic accessibility standards. Tailwind CSS's responsive utility classes ensure the application functions well on screens ranging from mobile phones to large desktop monitors.

The AI-powered receipt scanning feature specifically targets the most friction heavy part of expense tracking data entry and eliminates it through automation, extraction of form fields through AI along with proving insights and suggestion for user finance history every month.

D. Comparison with Existing Systems

Feature	Traditional App	FinSight AI	Spend Smart
Receipt scanning	Basic OCR only	OCR + NLP	Gemini multimodal
Monthly AI reports	Not available	Available	Available with Gemini
Background automation	None	Inngest	Inngest (cron + events)
Authentication	Basic	Clerk	Clerk with MFA
Bot/abuse protection	None	Arcjet	Arcjet middleware
Email communication	Basic	Resend	Resend + React Email
Visualization	Limited	Dashboard charts	Recharts interactive charts
Cloud file storage	None	Supabase	Supabase with signed URLs

E. Limitations and Future Work

Despite its capabilities, Spend Smart currently relies on manual account setup and does not support automatic bank feed integration through systems like Open Banking. it still requires human efforts to add their transaction into the application, user has to upload the digital receipt for scanning purpose.

Future work will focus on integrating financial data APIs that allow users to connect their bank accounts directly and handles the online transactions made without any manual efforts. This will enable fully automated transaction imports. The Gemini receipt scanning works well for printed and digital receipts, but it can produce inconsistent results for handwritten receipts. user paying through UPI make it easy for spend smart with literally zero efforts. though it also raise the concern of safety for monitoring the online transaction of the user, it must require the user allowance to monitor, ensuring the user user satisfaction regarding safety.

Future versions of Spend Smart will also look into predictive analytics, which uses historical transaction data to forecast future spending and savings patterns. Additionally, there will be a conversational AI interface that lets users ask about their financial data in natural language.

X. CONCLUSION

Spend Smart is an AI-powered expense tracking system that demonstrate how latest web- technologies and intelligent automation can improve personal finance management. Our proposed system Spend Smart integrates a full-stack architecture using Next.js, backend services, and database management to provide a simple and user friendly platform for monitoring income and expenditure. Key features like real-time transaction management, budget monitoring, data visualization through charts and automated alerts improve the usability and effectiveness of the application. Advanced features like AI-based receipt scanning, monthly financial insights generation, and automated email notifications strengthens the system by reducing the manual effort from user's end and make it more attractive toward users.

To ensure data integrity and user privacy, our system Spend Smart also implement security measures like authentication, rate

limiting, and bot protection. Additional features like use of scheduling mechanisms for recurring transactions and budget alerts highlights the capability of system to provide continuous financial assistance.

XI. REFRENCES

[1]. Rahmah Mahdi, *"Personal Expenses Tracker"*, Methodist University, 2024.
DOI: 10.13140/RG.2.2.20455.25760

[2]. Sneha Gupta, Unnati Jaiswal, Shubhangi Bhardwaj, Riya Bhargava,
"Personal Expense Tracker", International Journal of Novel Research and Development, Vol. 9, Issue 5, 2024.

[3]. Dr. Nidhi Sharma, Dr. Alok Sharma,
"Smart Personal Expense Tracker Technology", International Journal of Research and Analytical Reviews, Vol. 11, Issue 1, 2024.

[4]. Tushar Khuteta,
"Expense Tracker System (ETS)", IJPREMS, 2025.

[5]. A. Saraboji, Dr. Santhana Lakshmi,
"Expense Tracker Application", IARJSET, Vol. 11, Issue 6, 2024.

[6]. Lavesh Lingayat et al.,
"Design and Implementation of Real-Time Expense Tracker Using Machine Learning", ICICC Conference, 2024.

[7]. Pooja S. Saravade et al.,
"Income and Expense Daily Tracker System", IJIERT, 2022.

[8]. Mohammed Sahel et al.,
"Streamlining Personal Finances: An Expense Tracker Website Study", IEEE SPARC Conference, 2024.

[9]. (Multiple Authors),
"Expense Tracker using OCR and Random Forest Algorithm", IJERST, 2025.

[10]. Satyam Jha et al.,
"Design and Implementation of a Web-Based Expense Tracker System for Personal Finance Management", Zenodo, 2025.

PROOF OF SUBMISSION / ACCEPTANCE / PUBLICATION

Abstract Received
1 message

ICIEEST 2026 <icieest@iferp.net> Wed, Apr 29, 2026 at 17:43
To: aryan.2226csaiml1029@kiet.edu

Dear

We are delighted to inform you that we have received your article submission for the International Conference on Interdisciplinary Approaches in Education, Engineering & Social Transformation (ICIEEST – 2026), which will be held on May 22nd & 23rd, 2026, in Shanghai, China. The conference, Organized by IFERP Academy, will be themed "Transforming Education through Digital Innovation".

Next Steps:

To track and receive real-time updates on your abstract, please log in to our Organized by IFERP Academy, Conference Management System.

Please note that the review process typically takes 3-4 business days, and you will be informed of the status of your paper once it has been reviewed.

WhatsApp

PLAGIARISM REPORT

turnitin Page 2 of 9 - AI Writing Overview Submission ID trn:oid:::29034:136878470

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

turnitin Page 2 of 10 - Integrity Overview Submission ID trn:oid:::29034:136878470

3% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

- **15** Not Cited or Quoted 3%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

2% Internet sources
0% Publications
3% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.